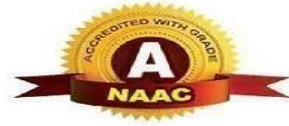


KANPUR INSTITUTE OF TECHNOLOGY



SOFTWARE ENGINEERING LAB FILE

SUBJECT CODE: BCS 651



Submitted by:

B.Tech (6thSem)

Submitted to:

Dr. Habib Ur Rahman
(PROFESSOR)

Academic Session 2025-26
COMPUTER SCIENCE DEPARTMENT

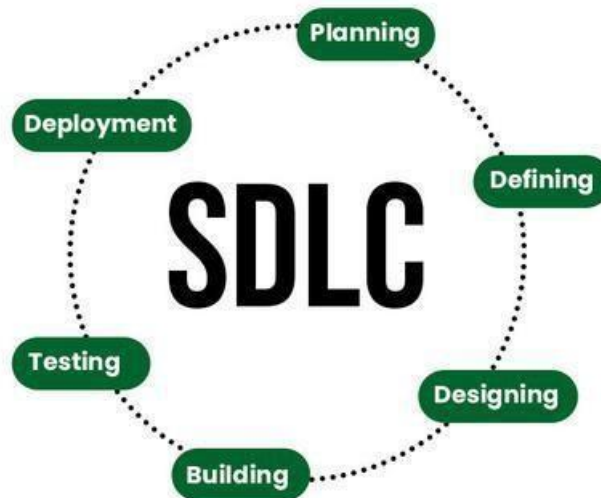
List of Experiments

Sr.NO	NameofProgram	Date	Sign

Experiment:-01

SoftwareDevelopmentLifeCycle

Software development lifecycle (SDLC) is a structured process that is used to design, develop, and test good-quality software. SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step.



The **goal of the SDLC life cycle model** is to deliver high-quality, maintainable software that meets the user's requirements. SDLC in software engineering models outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets users' requirements.

The [SDLC Model](#) involves six phases or stages while developing any software.

Stage-1: Planning and Requirement Analyze

Planning is a crucial step in everything, just as in [software development](#). In this same stage, [requirement analysis](#) is also performed by the developers of the organization. This is attained from customer inputs, and sales department/market surveys.

The information from this analysis forms the building blocks of a basic project. The quality of the project is a result of planning. Thus, in this stage, the basic project is designed with all the available information.

Stage-2: Defining Requirements

In this stage, all the requirements for the target software are specified. These requirements get approval from customers, market analysts, and stakeholders. This is fulfilled by utilizing SRS (Software Requirement Specification). This is a sort of document that specifies all those things that need to be defined and created during the entire project cycle.

Stage-3:DesigningArchitecture

[SRS](#) is a reference for software designers to come up with the best architecture for the software. Hence, with the requirements defined in SRS, multiple designs for the product architecture are present in the Design Document Specification (DDS).

This DDS is assessed by market analysts and stakeholders. After evaluating all the possible factors, the most practical and logical design is chosen for development.

Stage-4:DevelopingProduct

At this stage, the fundamental development of the product starts. For this, developers use a specific programming code as per the design in the DDS. Hence, it is important for the coders to follow the protocols set by the association. Conventional programming tools like compilers, interpreters, debuggers, etc. are also put into use at this stage. Some popular languages like C/C++, Python, Java, etc. are put into use as per the software regulations.

Stage-5:ProductTestingandIntegration

After the development of the product, testing of the software is necessary to ensure its smooth execution. Although, minimal testing is conducted at every stage of SDLC. Therefore, at this stage, all the probable flaws are tracked, fixed, and retested. This ensures that the product confronts the quality requirements of SRS.

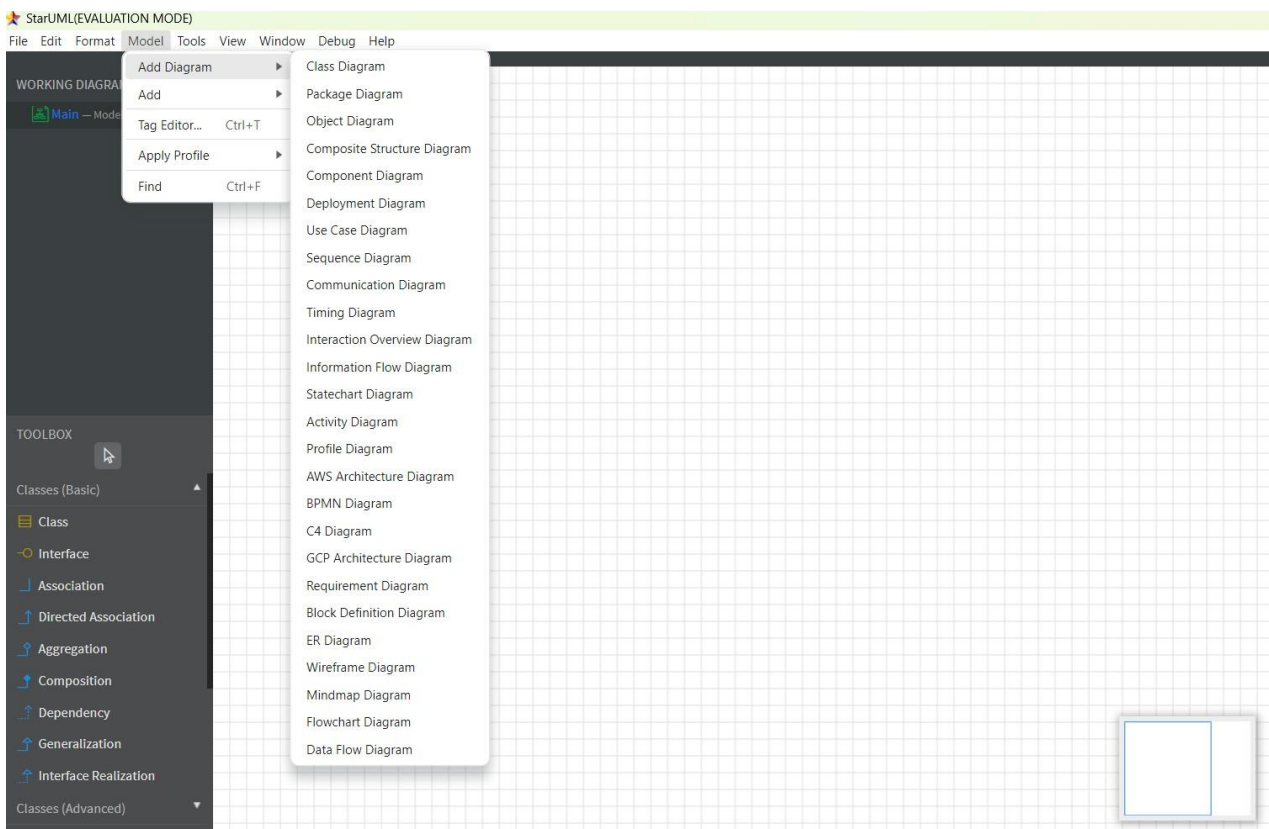
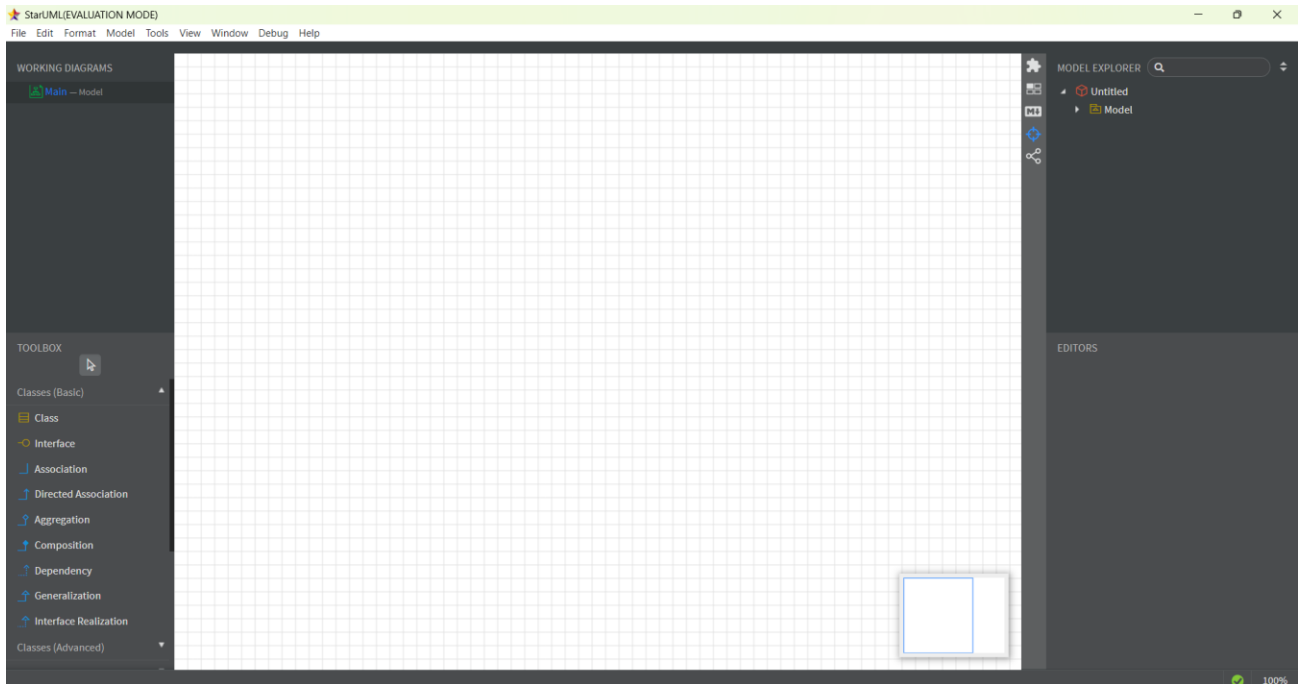
Documentation, Training, and Support: [Software documentation](#) is an essential part of the software development life cycle. A well-written document acts as a tool and means to information repository necessary to know about software processes, functions, and maintenance. Documentation also provides information about how to use the product. Training is an attempt to improve the current or future employee performance by increasing an employee's ability to work through learning, usually by changing his attitude and developing his skills and understanding.

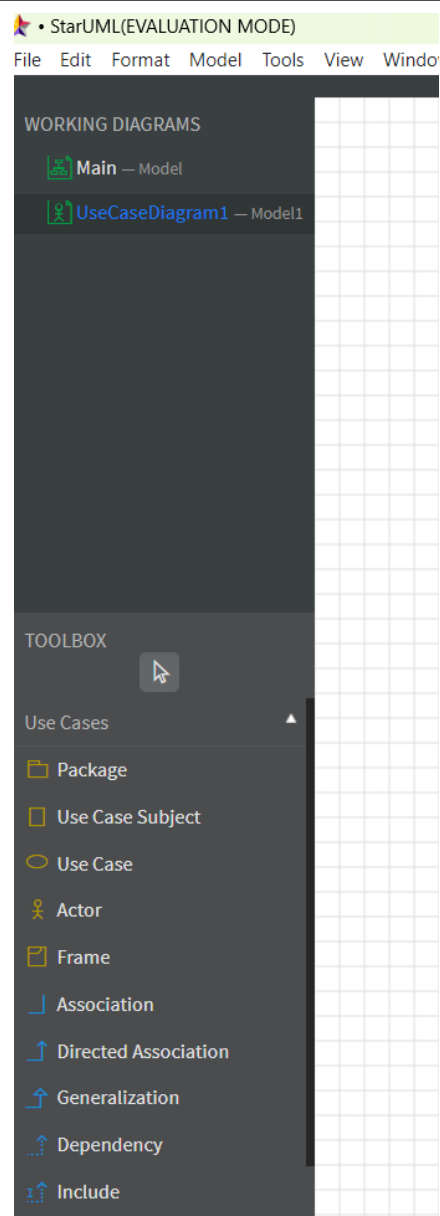
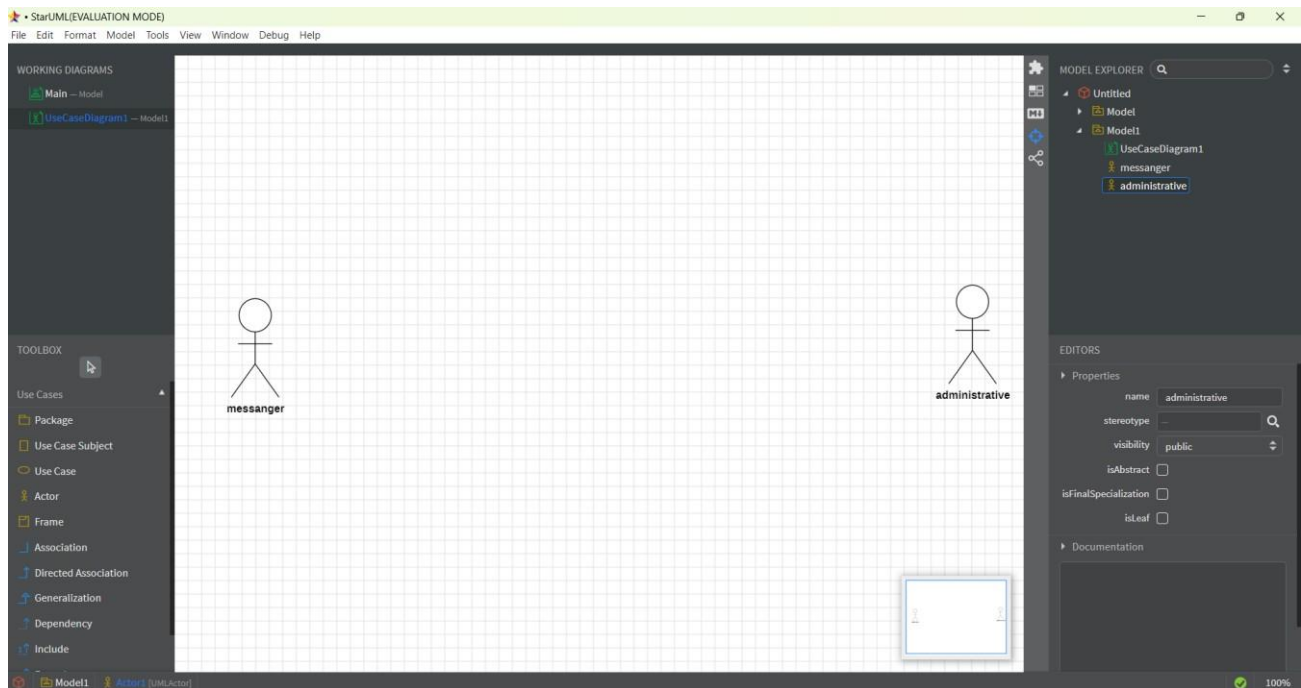
Stage-6:DeploymentandMaintenanceofProducts

After detailed testing, the conclusive product is released in phases as per the organization's strategy. Then it is tested in a real industrial environment. It is important to ensure its smooth performance. If it performs well, the organization sends out the product as a whole. After retrieving beneficial feedback, the company releases it as it is or with auxiliary improvements to make it further helpful for the customers. However, this alone is not enough. Therefore, along with the deployment, the [product's supervision](#).

Experiment:-02

Screenshots of StarUML





Experiment:-03

Software Requirement Specifications

The production of the requirements stage of the software development process is **Software Requirements Specifications (SRS)** (also called a **requirements document**). This report lays a foundation for software engineering activities and is constructed when entire requirements are elicited and analyzed. SRS is a formal report, which acts as a representation of software that enables the customers to review whether it (SRS) is according to their requirements. Also, it comprises user requirements for a system as well as detailed specifications of the system requirements.

The SRS is a specification for a specific software product, program, or set of applications that perform particular functions in a specific environment. It serves several goals depending on who is writing it. First, the SRS could be written by the client of a system. Second, the SRS could be written by a developer of the system. The two methods create entirely various situations and establish different purposes for the document altogether. The first case, SRS, is used to define the needs and expectation of the users. The second case, SRS, is written for various purposes and serves as a contract document between customer and developer.

Template of SRS: -

1. Introduction

- **Purpose of this Document** – At first, main aim of why this document is necessary and what's purpose of document is explained and described.
- **Scope of this document** – In this, overall working and main objective of document and what value it will provide to customer is described and explained. It also includes a description of development cost and time required.
- **Overview** – In this, description of product is explained. It's simply summary or overall review of product.

1. General description

In this, general functions of product which includes objective of user, a user characteristic, features, benefits, about why its importance is mentioned. It also describes features of user community.

2. Functional Requirements

In this, possible outcome of software system which includes effects due to operation of program is fully explained. All functional requirements which may include calculations, data processing, etc. are placed in a ranked order. Functional requirements specify the expected behaviour of the system - which outputs should be produced from the given inputs.

They describe the relationship between the input and output of the system. For each functional requirement, detailed description all the data inputs and their source, the units of measure, and the range of valid inputs must be specified.

3. InterfaceRequirements

In this, software interfaces which mean how software program communicates with each other or user either in form of any language, code, or message are fully described and explained. Examples can be shared memory, data streams, etc.

4. Performance Requirements

In this, how a software system performs desired functions under specific condition is explained. It also explains required time, required memory, maximum error rate, etc. The performance requirements part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified. There are two types of performance requirements: static and dynamic. Static requirements are those that do not impose constraint on the execution characteristics of the system. Dynamic requirements specify constraints on the execution behaviour of the system.

5. Design Constraints

In this, constraints which simply means limitation or restriction are specified and explained for design team. Examples may include use of a particular algorithm, hardware and software limitations, etc. There are a number of factors in the client's environment that may restrict the choices of a designer leading to design constraints such factors include standards that must be followed, resource limits, operating environment, reliability and security requirements and policies that may have an impact on the design of the system. An SRS should identify and specify all such constraints.

6. Non-Functional Attributes

In this, non-functional attributes are explained that are required by software system for better performance. An example may include Security, Portability, Reliability, Reusability, Application compatibility, Data integrity, Scalability capacity, etc.

7. Preliminary Schedule and Budget

In this, initial version and budget of project plan are explained which include overall time duration required and overall cost required for development of project.

8. Appendices

In this, additional information like references from where information is gathered, definitions of some specific terms, acronyms, abbreviations, etc. are given and explained.

Experiment:-04

E-RDiagram

Peter Chen developed the ER diagram in 1976. The ER model was created to provide a simple and understandable model for representing the structure and logic of databases. It has since evolved into variations such as the Enhanced ER Model and the Object Relationship Model.

The Entity Relational Model is a model for identifying entities to be represented in the database and representation of how those entities are related. The ER data model specifies enterprise schema that represents the overall logical structure of a database graphically.

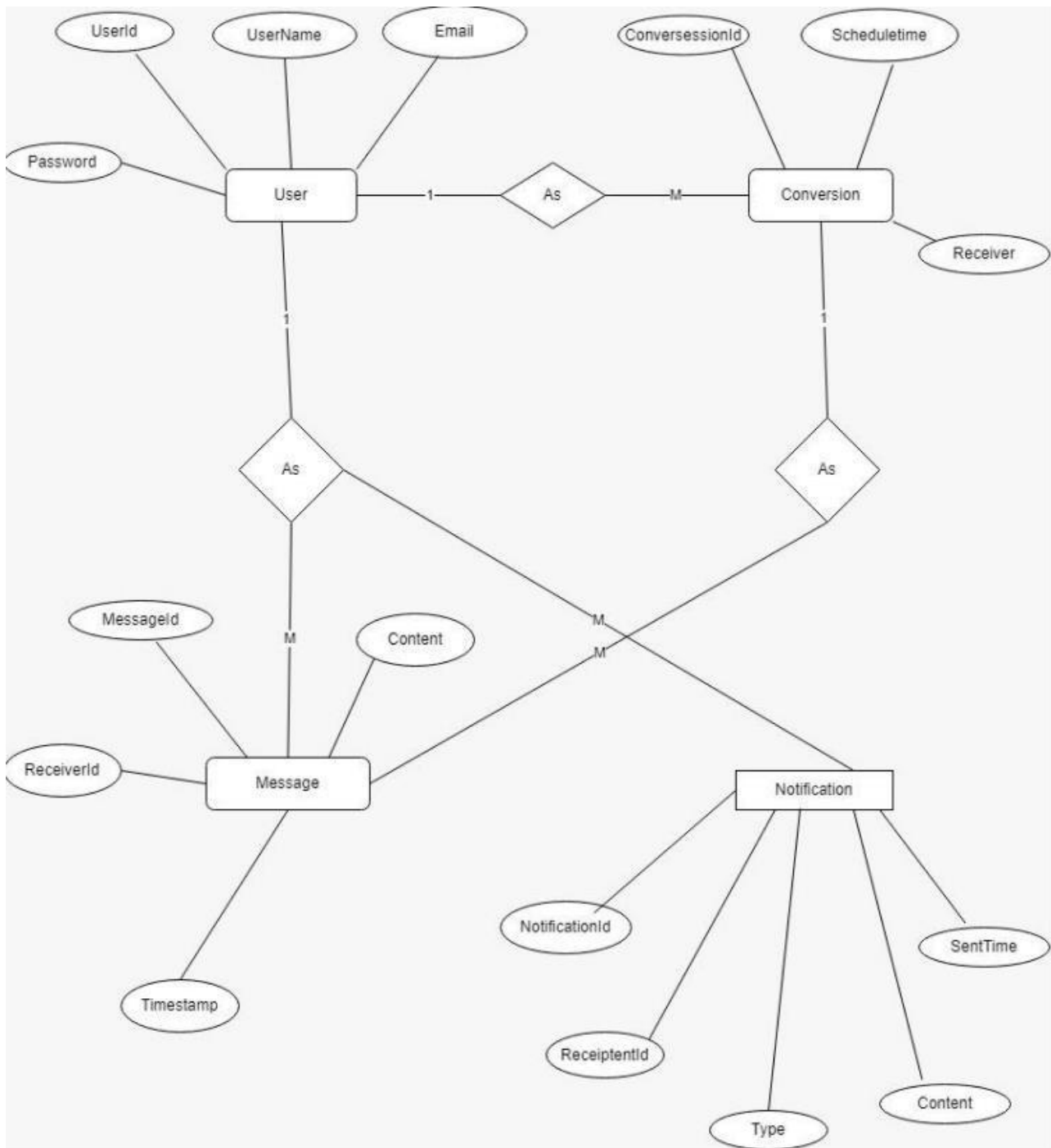
The Entity Relationship Diagram explains the relationship among the entities present in the database. ER models are used to model real-world objects like a person, a car, or a company and the relation between these real-world objects. In short, the ER Diagram is the structural format of the database.

Symbols Used in ER Model

ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

- **Rectangles:** Rectangles represent Entities in the ER Model.
- **Ellipses:** Ellipses represent Attributes in the ER Model.
- **Diamond:** Diamonds represent Relationships among Entities.
- **Lines:** Lines represent attributes to entities and entity sets with other relationship types.
- **Double Ellipse:** Double Ellipses represent Multi-Valued Attributes.
- **Double Rectangle:** Double Rectangle represents a Weak Entity.

E-R Diagram of Easy chat



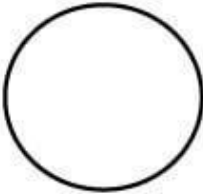
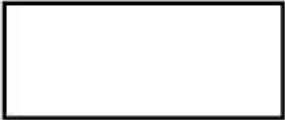
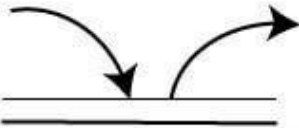
Experiment:-05

DFDdiagram

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both.

It shows how data enters and leaves the system, what changes the information, and where data is stored.

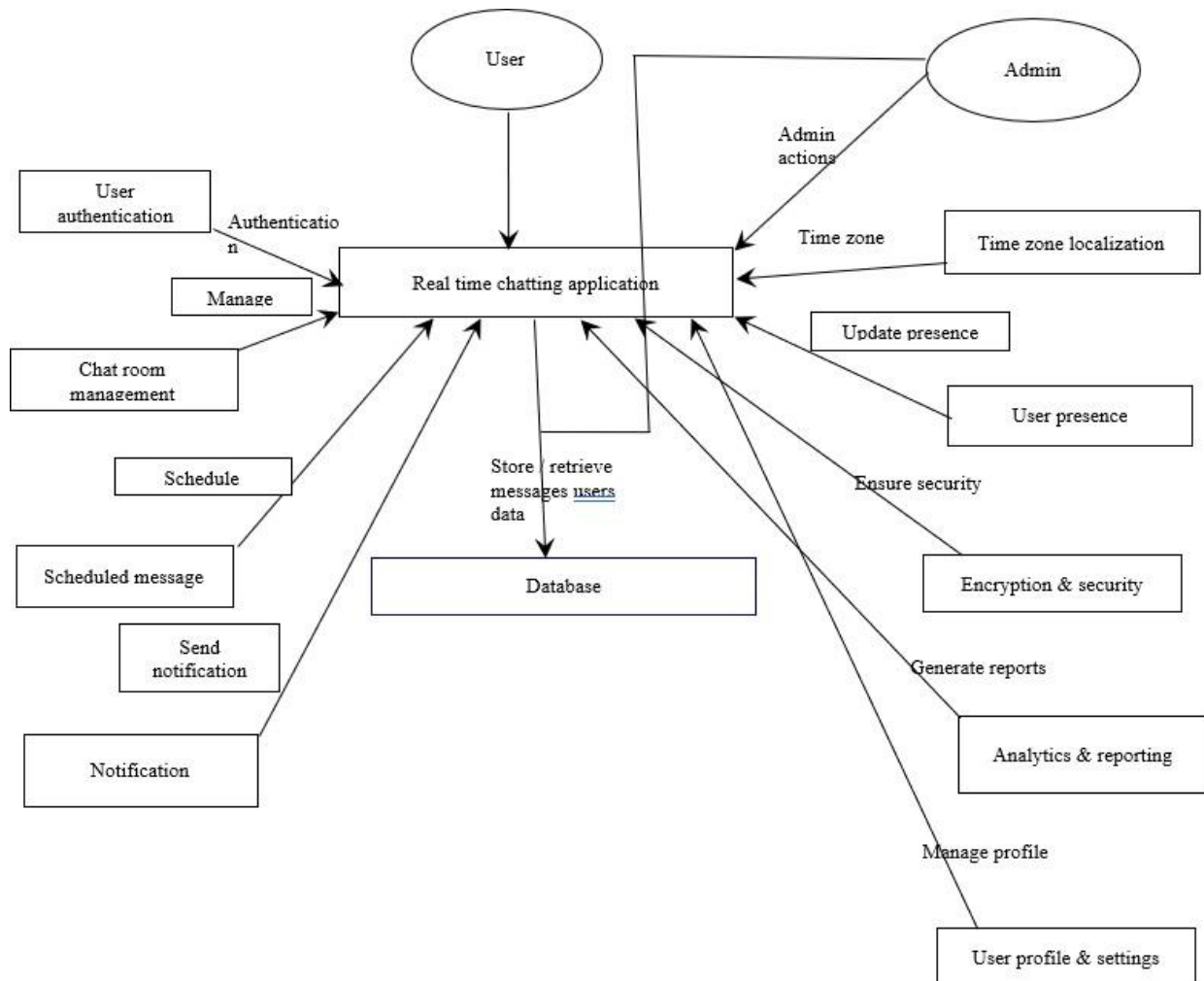
The objective of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

Symbol	Name	Function
	Data flow	Used to Connect Processes to each other, to sources or Sinks; the arrow head indicates direction of data flow.
	Process	Performs Some transformation of Input data to yield output data.
	Source of Sink (External Entity)	A Source of System inputs or Sink of System outputs.
	Data Store	A repository of data; the arrow heads indicate net inputs and net outputs to store.

Symbols for Data Flow Diagrams

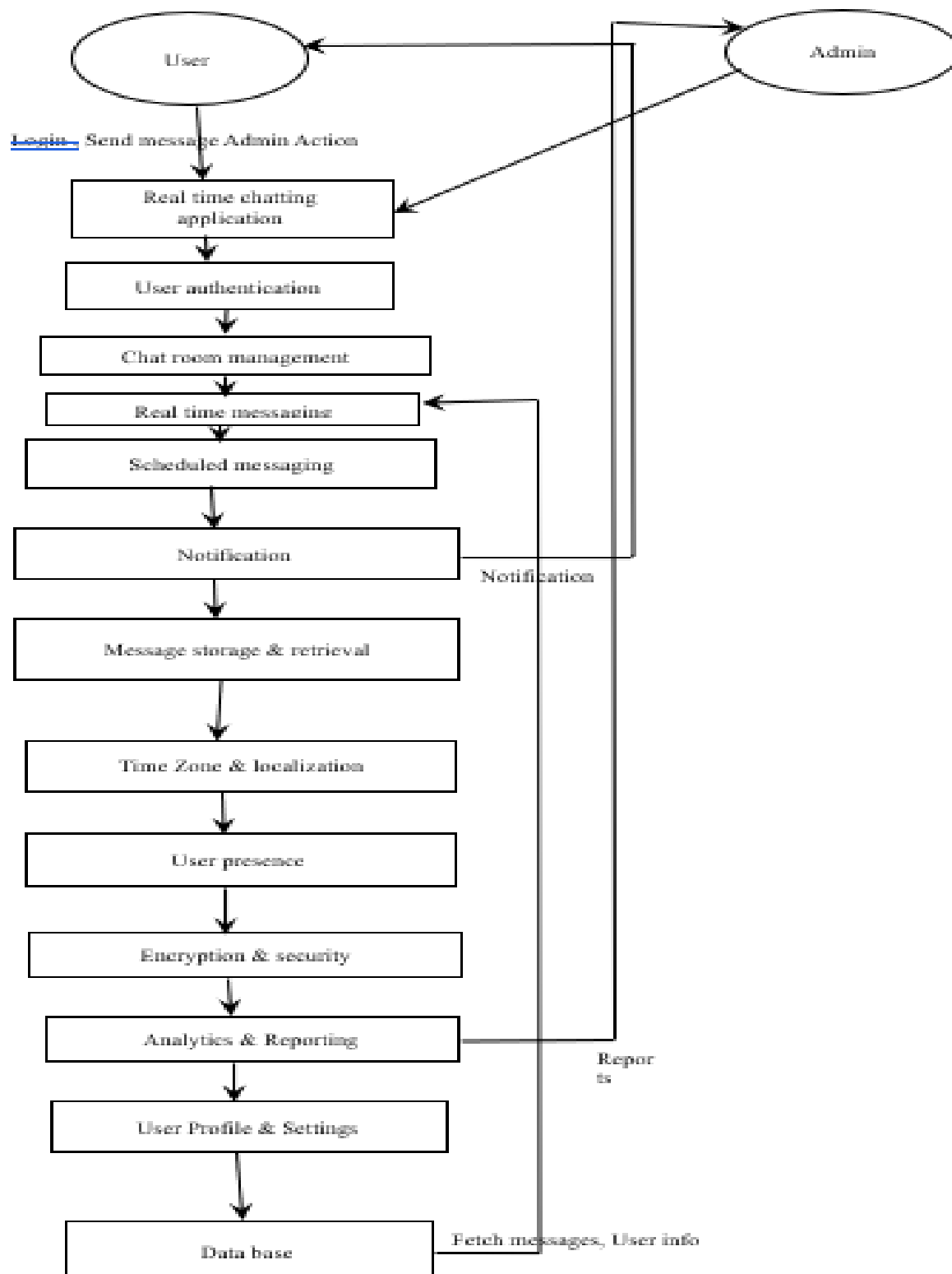
0 Level DFD Easy Chat

It is also known as a context diagram. It's designed to be an abstraction view, showing the system as a single process with its relationship to external entities. It represents the entire system as a single bubble with input and output data indicated by incoming/outgoing arrows.



1 LevelDFD of Easy Chat

This level provides a more detailed view of the system by breaking down the major processes identified in the level 0 DFD into sub-processes. Each sub-process is depicted as a separate process on the level 1 DFD. The data flows and data stores associated with each sub-process are also shown. In 1-level DFD, the context diagram is decomposed into multiple bubbles/processes. In this level, we highlight the main functions of the system and breakdown the high-level process of 0-level DFD into subprocesses.



Experiment:-06

Use case diagram

A use case diagram is used to represent the dynamic behaviour of a system. It encapsulates the system's functionality by incorporating use cases, actors, and their relationships. It models the tasks, services, and functions required by a system/subsystem of an application. It depicts the high-level functionality of a system and also tells how the user handles a system.

Use Case Diagram Notations

UML notations provide a visual language that enables software developers, designers, and other stakeholders to communicate and document system designs, architectures, and behaviours in a consistent and understandable manner.

1. Actors

Actors are external entities that interact with the system. These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately modelling system behavior.

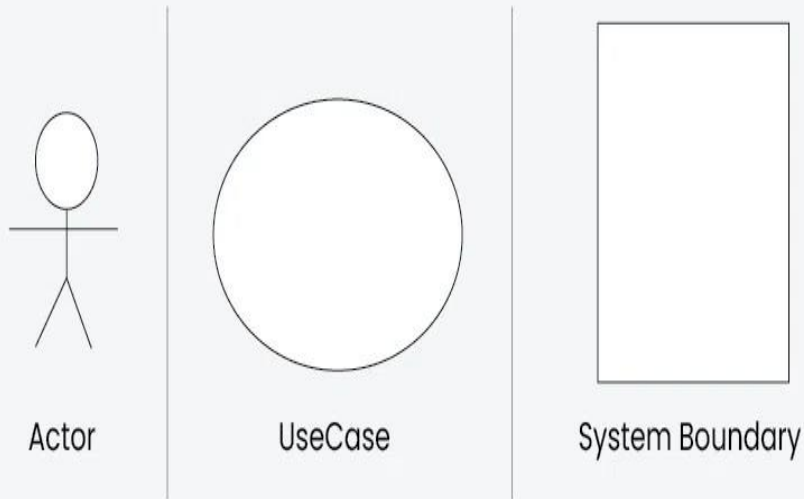
2. Use Cases

Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be "Place Order," "Track Delivery," or "Update Product Information". Use cases are represented by ovals.

3. System Boundary

The system boundary is a visual representation of the scope or limits of the system you are modelling. It defines what is inside the system and what is outside. The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.

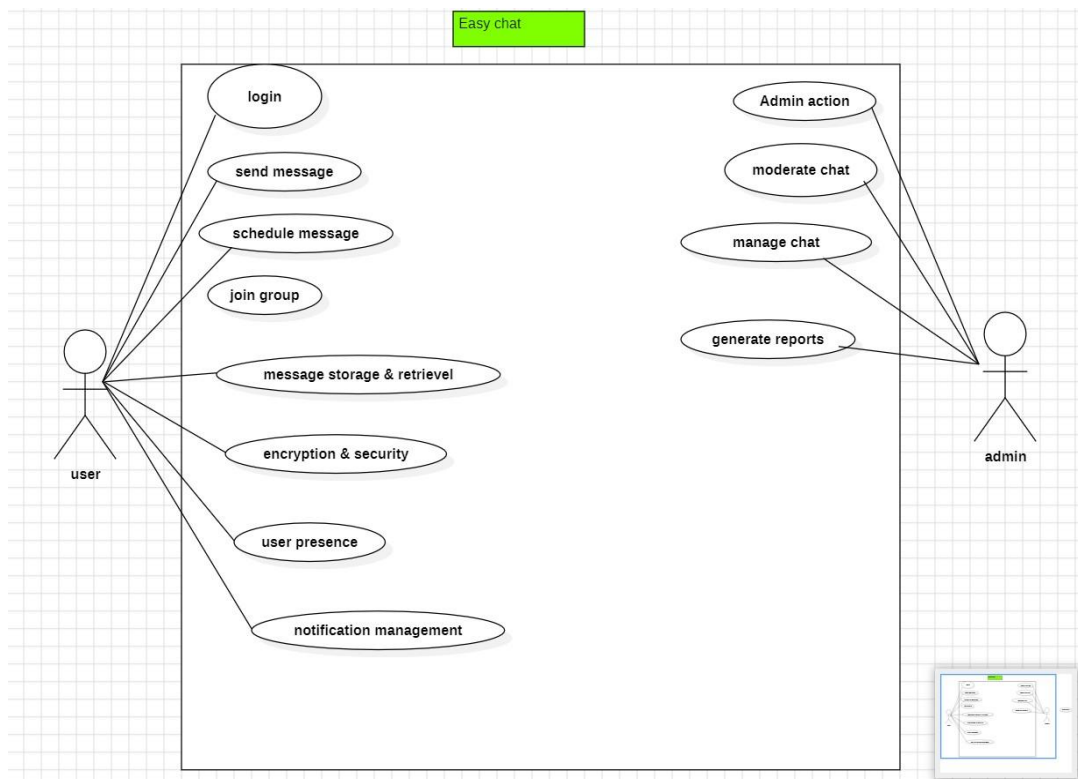
Use Case Diagram Notations



Use Case Diagrams | Unified Modeling Language (UML)



Use case diagram of Easy chat



Experiment:-07

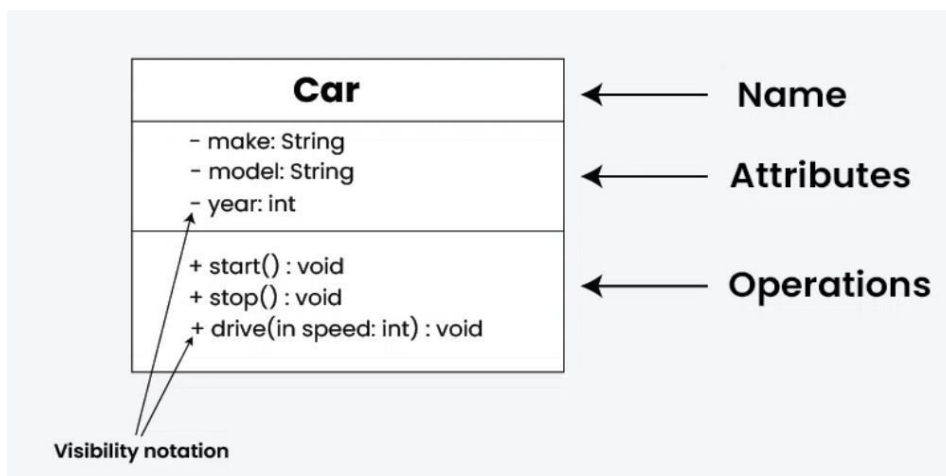
Classdiagram

The class diagram depicts a static view of an application. It represents the types of objects residing in the system and the relationships between them. A class consists of its objects, and also it may inherit from other classes. A class diagram is used to visualize, describe, document various different aspects of the system, and also to construct executable software code.

It shows the attributes, classes, functions, and relationships to give an overview of the software system. It constitutes class names, attributes, and functions in a separate compartment that helps in software development. Since it is a collection of classes, interfaces, associations, collaborations, and constraints, it is termed as a structural diagram.

UML Class Notation

Class notation is a graphical representation used to depict classes and their relationships in object-oriented modelling.



1. **ClassName:**

- The name of the class is typically written in the top compartment of the class box and is centred and bold.

2. **Attributes:**

- Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.

3. **Methods:**

- Methods, also known as functions or operations, represent the behaviour or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.

4. **Visibility Notation:**

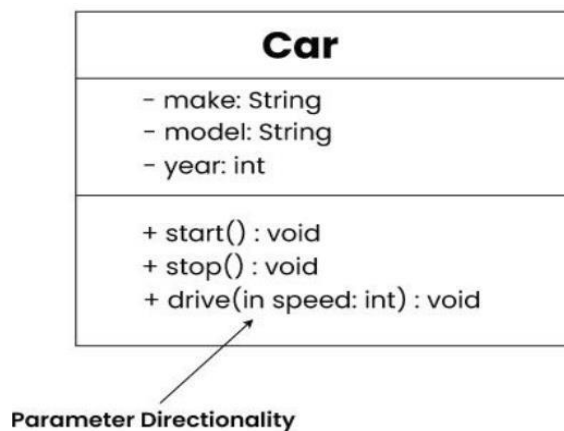
- Visibility notations indicate the access level of attributes and methods. Common visibility

notations include:

- + for public (visible to all classes)
- - for private (visible only within the class)
- # for protected (visible to subclasses)
- ~ for package or default visibility (visible to classes in the same package)

Parameter Directionality

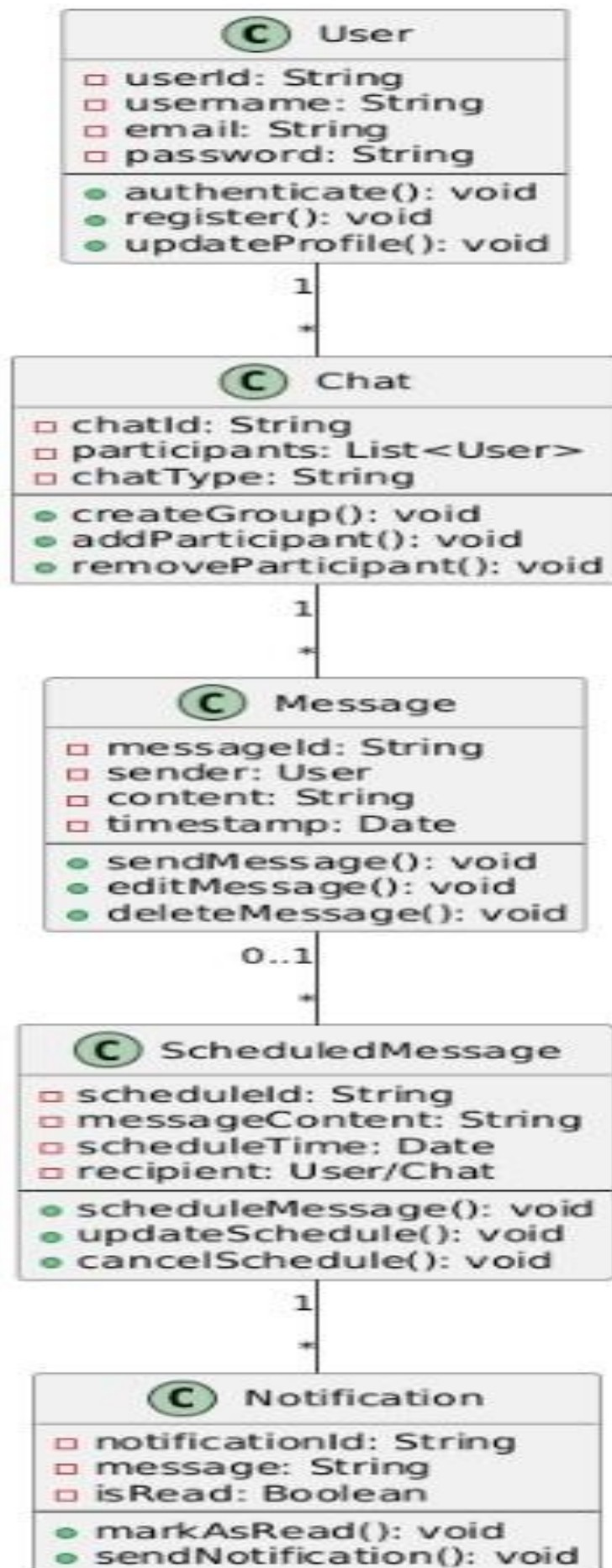
In class diagrams, parameter directionality refers to the indication of the flow of information between classes through method parameters. It helps to specify whether a parameter is an input, an output, or both. This information is crucial for understanding how data is passed between objects during method calls.



There are three main parameter directionality notations used in class diagrams:

- **In(Input):**
 - An input parameter is a parameter passed from the calling object (client) to the called object (server) during a method invocation.
 - It is represented by an arrow pointing towards the receiving class (the class that owns the method).
- **Out(Output):**
 - An output parameter is a parameter passed from the called object (server) back to the calling object (client) after the method execution.
 - It is represented by an arrow pointing away from the receiving class.
- **InOut(Input and Output):**
 - An InOut parameter serves as both input and output. It carries information from the calling object to the called object and vice versa.
 - It is represented by an arrow pointing towards and away from the receiving class.

ClassdiagramofEasychat



Experiment:-08

Sequencediagram

A SequenceDiagram is a key component of UnifiedModellingLanguage(UML) used to visualize the interaction between objects in a sequential order.

SequenceDiagramNotations

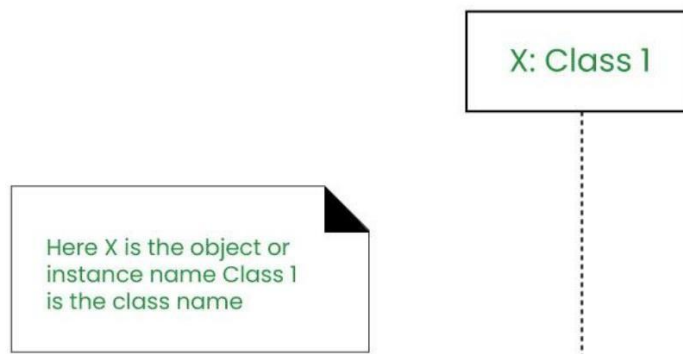
1. Actors

An actor in a UML diagram represents a type of role where it interacts with the system and its objects. It is important to note here that an actor is always outside the scope of the system we aim to model using the UML diagram.



2. Lifelines

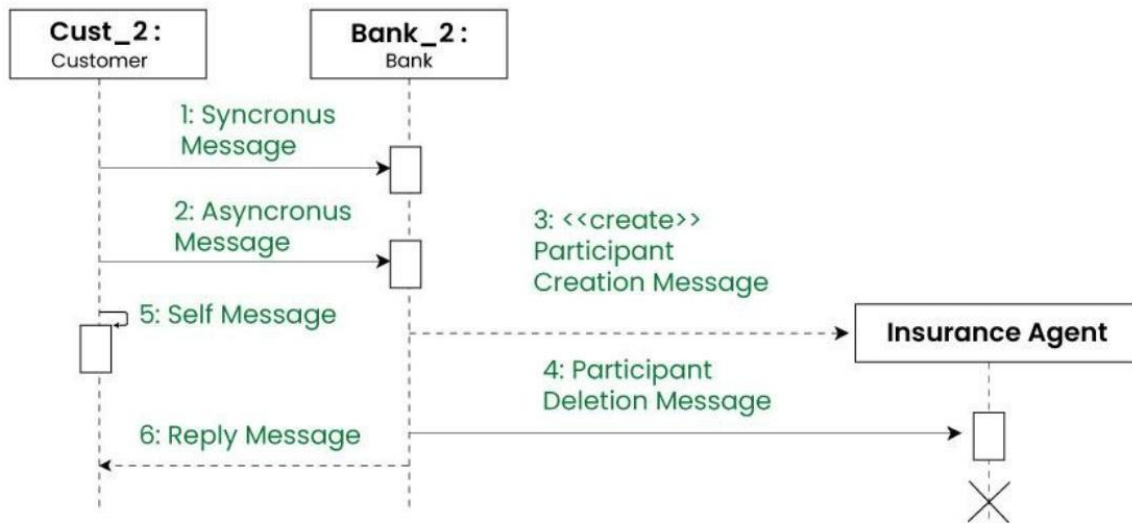
A lifeline is a named element which depicts an individual participant in a sequence diagram. So basically, each instance in a sequence diagram is represented by a lifeline.



3. Messages

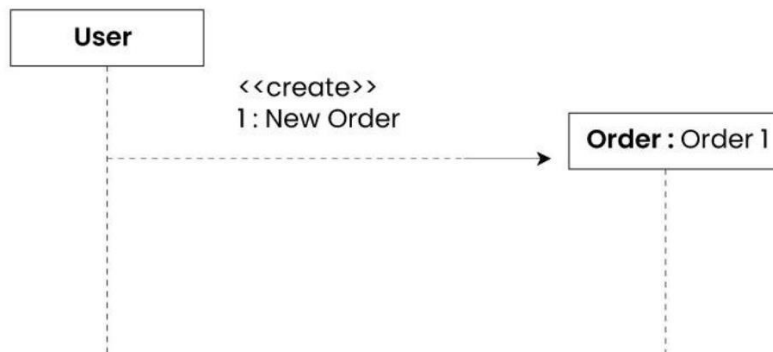
Communication between objects is depicted using messages. The messages appear in a sequential order on the lifeline.

- We represent messages using arrows.
- Lifelines and messages form the core of a sequence diagram.



4. Create message

We use a Create message to instantiate a new object in the sequence diagram. There are situations when a particular message call requires the creation of an object. It is represented with a dotted arrow and create word labelled on it to specify that it is the create message symbol.



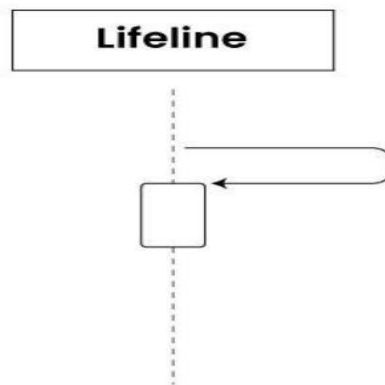
5. Delete Message

We use a Delete Message to delete an object. When an object is deallocated memory or is destroyed within the system we use the Delete Message symbol. It is represented by an arrow terminating with a cross.



6. Self-Message

Certain scenarios might arise where the object needs to send a message to itself. Such messages are called **Self Messages** and are represented with a **U-shaped arrow**.



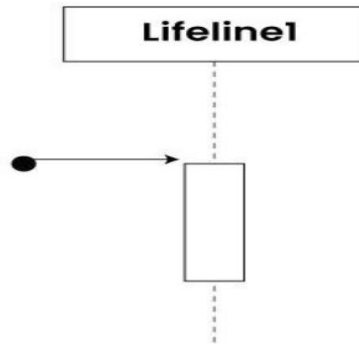
7. ReplyMessage

Reply messages are used to show the message being sent from the receiver to the sender. We represent a return/reply message using an **open arrow head with a dotted line**. The interaction moves forward only when a reply message is sent by the receiver.



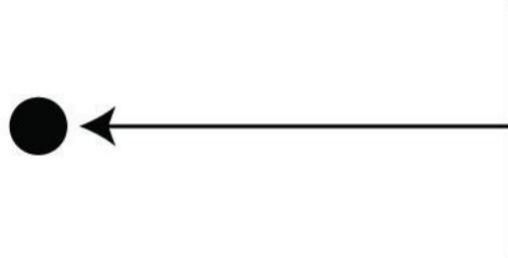
8. FoundMessage

A Found message is used to represent a scenario where an unknown source sends the message. It is represented using an **arrow directed towards a lifeline from an endpoint**.



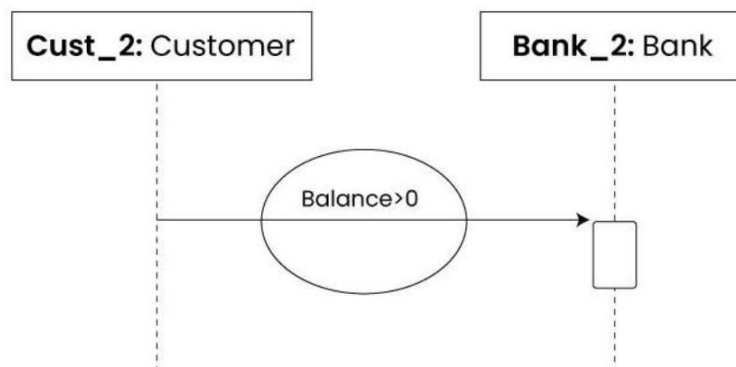
9. LostMessage

A Lost message is used to represent a scenario where the recipient is not known to the system. It is represented using an arrow directed towards an end point from a lifeline.

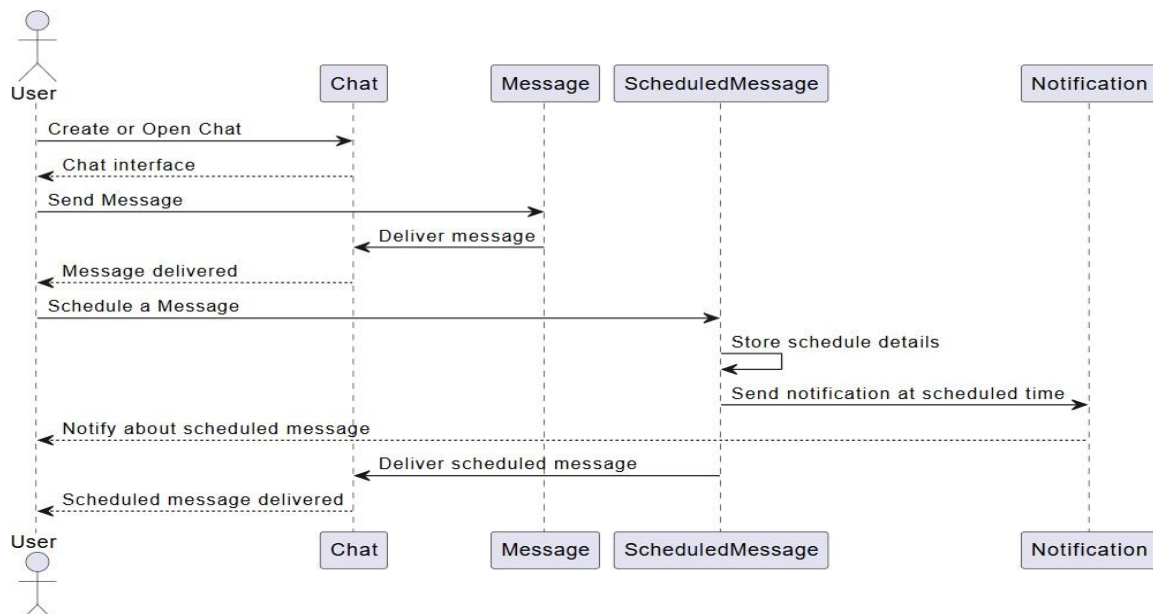


10. Guards

To model conditions, we use guards in UML. They are used when we need to restrict the flow of messages on the pretext of a condition being met.



Sequence Diagram of Easy Chat



Experiment:-09

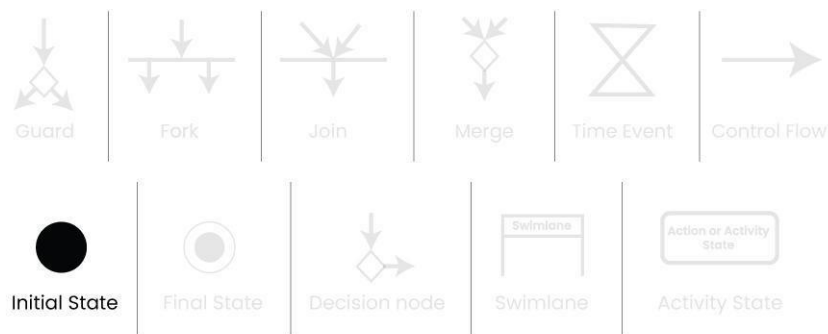
Activitydiagram

- Anactivitydiagramstartsfromaninitialpointandendsatafinalpoint,showingdifferentdecisionpathsalong theway.
- Theyareoftenusedinbusinessandprocessmodellingtoshowhowasystembehavesovertime.

ActivityDiagram Notations

1. InitialState

Thestartingstatebeforeanactivitytakesplaceisdepictedusingtheinitialstate.

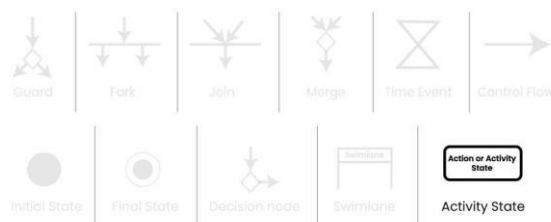


Initial State



2. ActionorActivityState

An activity represents execution of an action on objects or by objects. We represent an activityusing a rectangle with rounded corners. Basically, any action or event that takes place isrepresented usinganactivity.

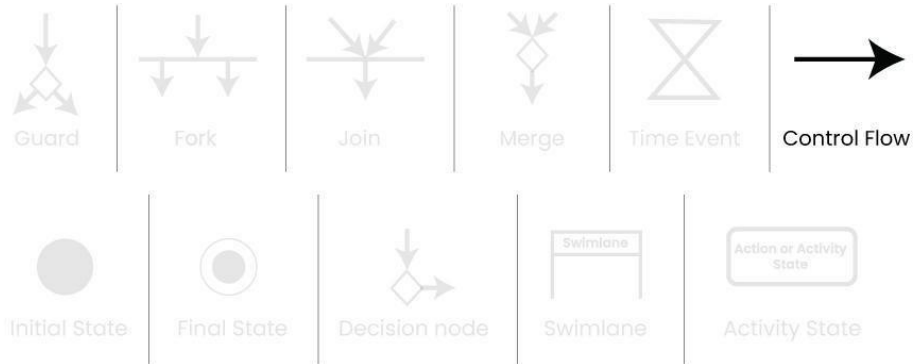


Activity State



3. ActionFloworControlflows

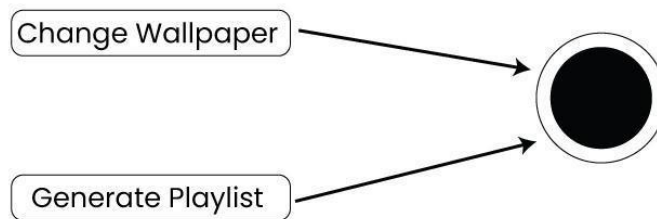
ActionflowsorControlflowsarealsoreferredtoaspathsandedges.Theyareusedtoshowthetransitionfromoneactivity statetoanotheractivitystate.



Control Flow



Using Action Flows for Transitions

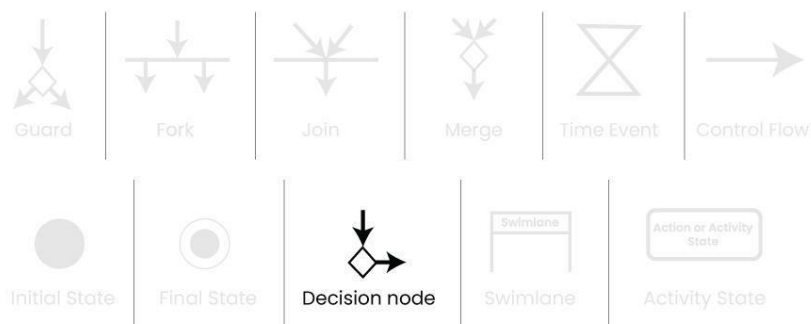


Unified Modeling Language (UML) | Activity Diagrams



4. Decision node and Branching

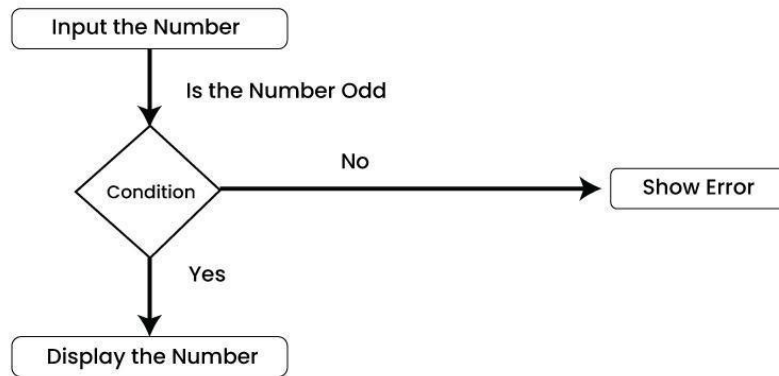
When we need to decide before deciding the flow of control, we use the decision node. The outgoing arrows from the decision node can be labelled with conditions or guard expressions. It always includes two or more output arrows.



Decision Node



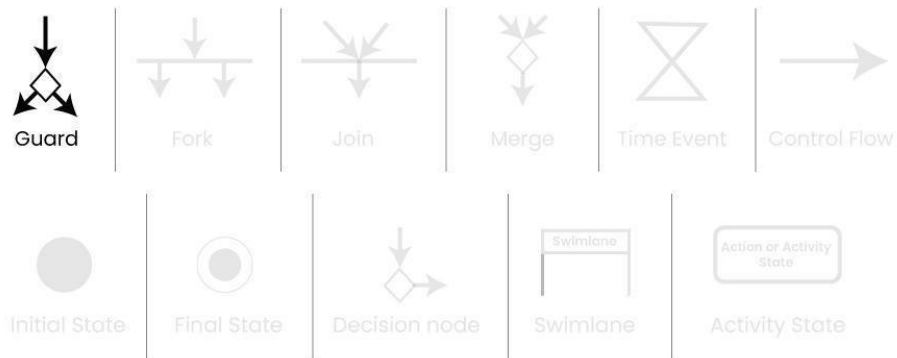
An Activity Diagram using Decision Node



Unified Modeling Language (UML) | Activity Diagrams



5. Guard



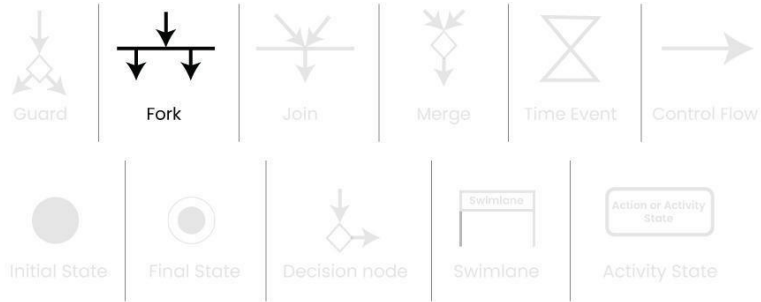
Guard



The statement must be true for the control to shift along a particular direction. Guards help us know the constraints and conditions which determine the flow of a process.

6. Fork

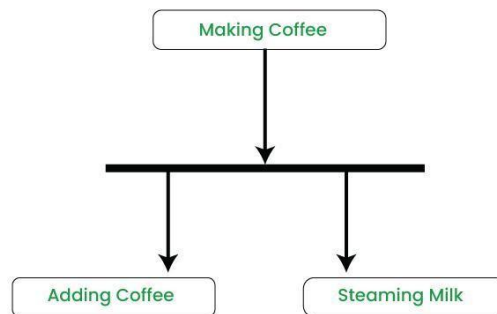
Fork nodes are used to support concurrent activities. When we use a fork node when both the activities get executed concurrently i.e. no decision is made before splitting the activity into two parts. Both parts need to be executed in case of a fork statement. We use a rounded solid rectangular bar to represent a Fork notation with incoming arrow from the parent activity state and outgoing arrows towards the newly created activities.



Fork



A Diagram using Fork Notation

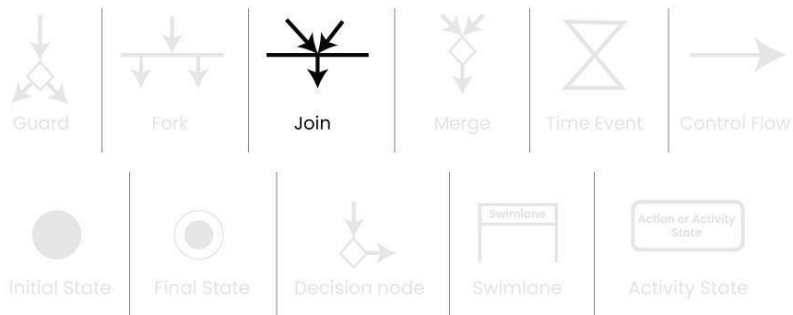


Unified Modeling Language (UML) | Activity Diagrams



7. Join

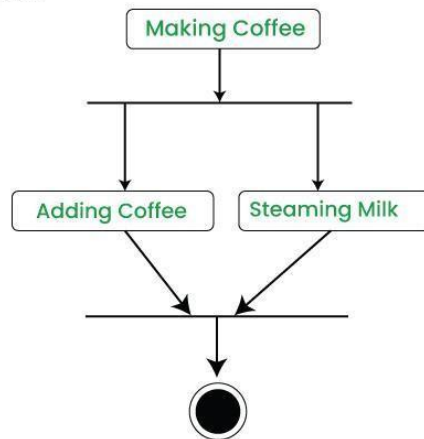
Join nodes are used to support concurrent activities converging into one. For join notations we have two or more incoming edges and one outgoing edge.



Join

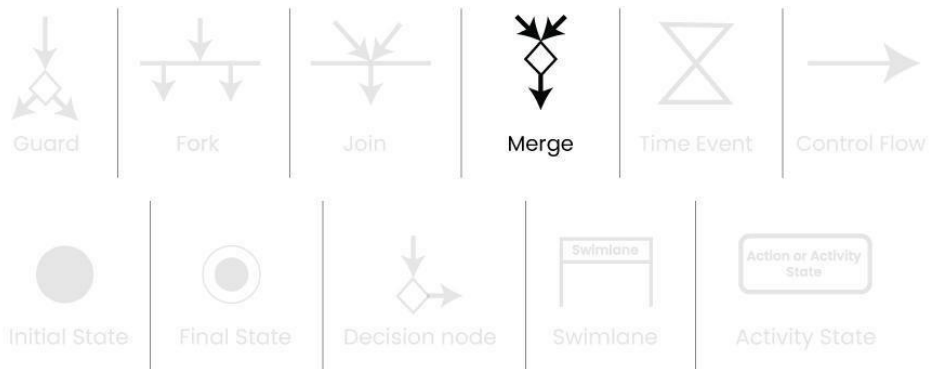


A Diagram using Join Notation



8. Merge or MergeEvent

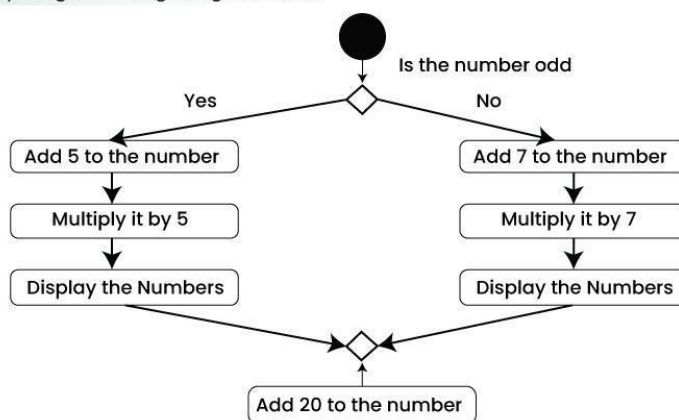
Scenarios arise when activities which are not being executed concurrently have to be merged. We use the merge notation for such scenarios. We can merge two or more activities into one if the control proceeds to the next activity irrespective of the path chosen.



Merge

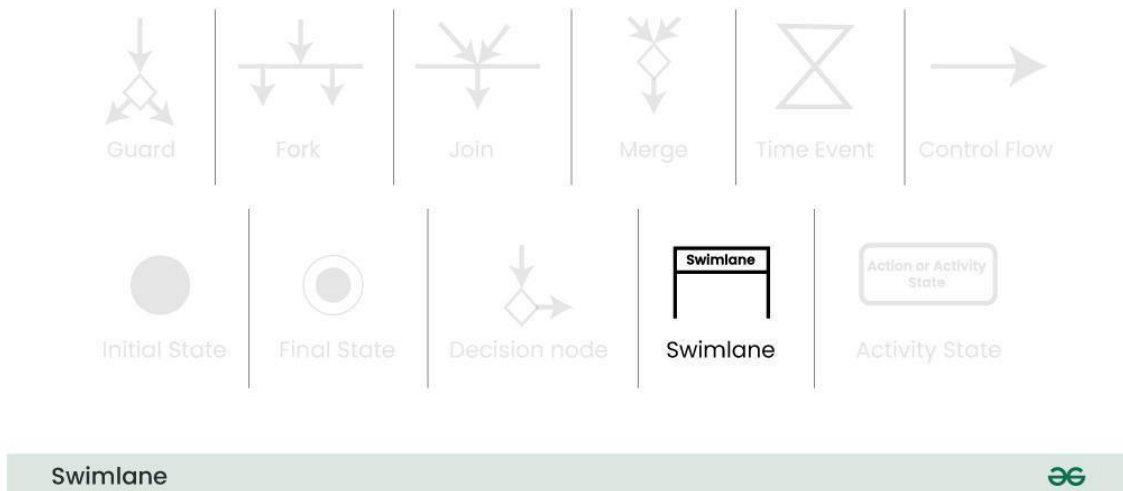


An Activity Diagram using Merge Notation



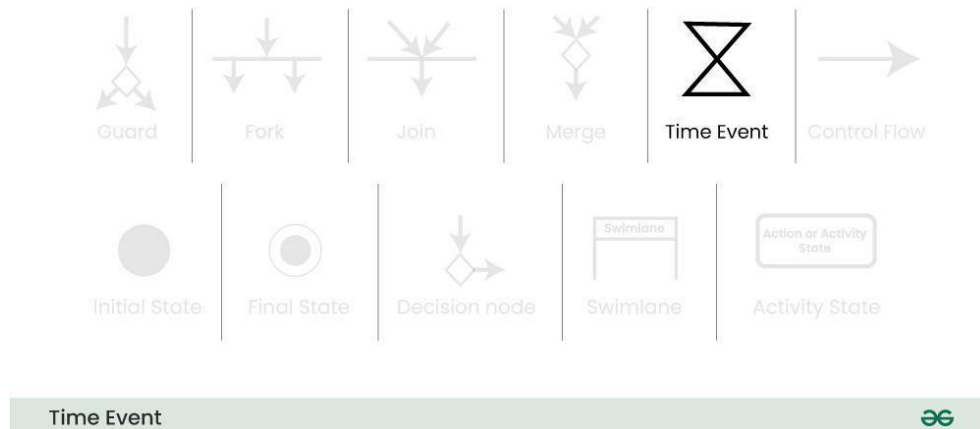
9. Swimlanes

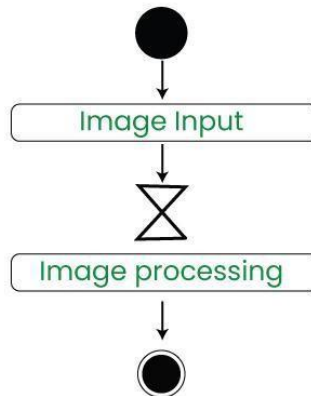
We use swimlanes for grouping related activities in one column. Swimlanes group related activities into one column or one row. Swimlanes can be vertical and horizontal. Swimlanes are used to add modularity to the activity diagram. It is not mandatory to use swimlanes. They usually give more clarity to the activity diagram. It's similar to creating a function in a program. It's not mandatory to do so, but, it is a recommended practice.



10. TimeEvent

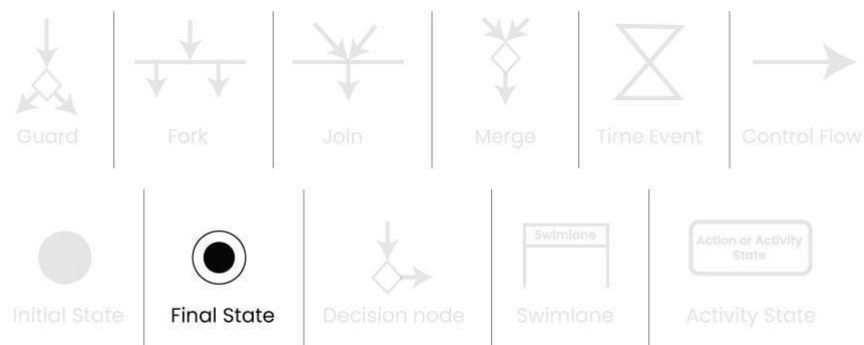
This refers to an event that stops the flow for a time; an hourglass depicts it. We can have a scenario where an event takes some time to be completed.



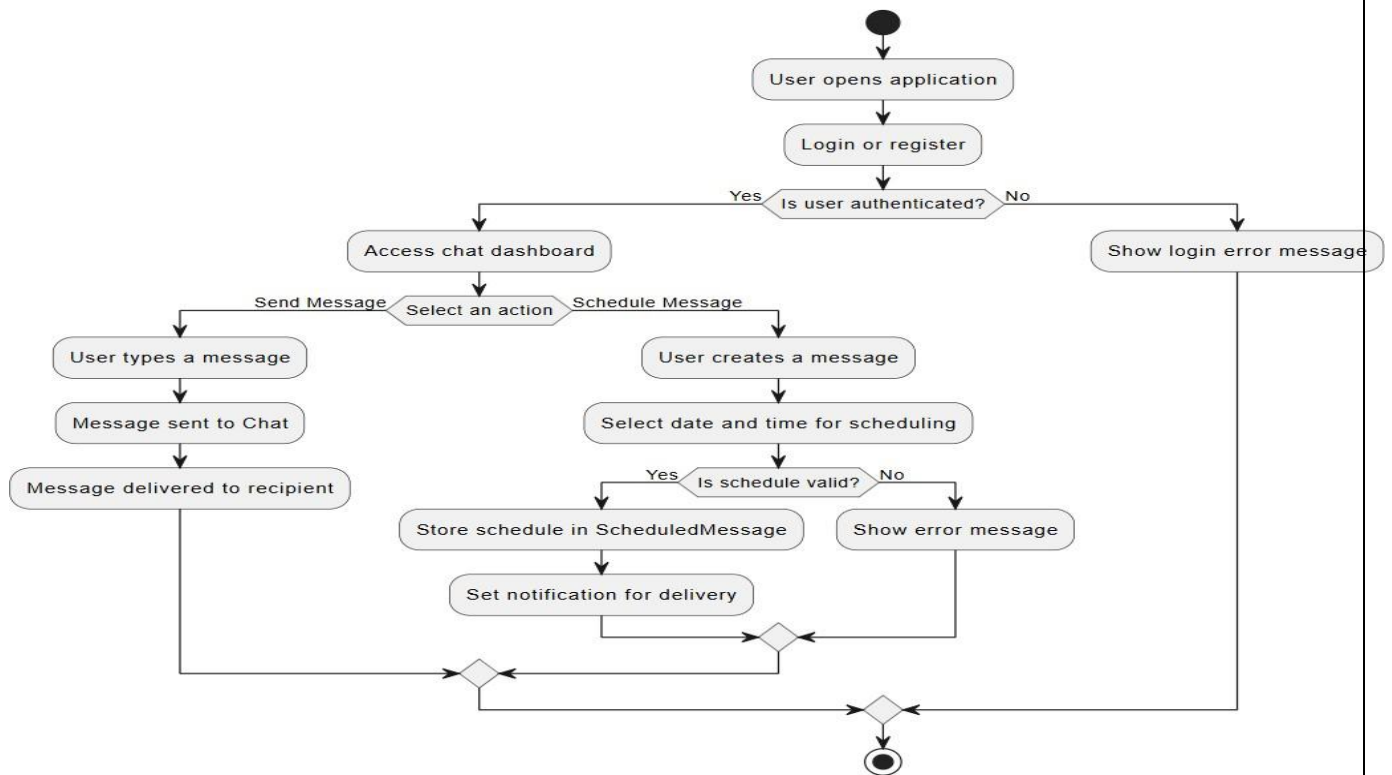


11. FinalStateorEndState

The state which the system reaches when a particular process or activity ends is known as a FinalState or End State. We use a filled circle within a circle notation to represent the final state in a statemachine diagram. A system or a process can have multiple final states.



Activitydiagram of Easy Chat



Experiment:-10

Deployment diagram

A Deployment Diagram shows how the software design turns into the actual physical system where the software will run. They show where software components are placed on hardware devices and show how they connect with each other.

Components and Notations in Deployment Diagram

Below are the components and their notations in deployment diagram:

1. Component



A component represents a modular and reusable part of a system, typically implemented as a software module, class, or package. It encapsulates its behaviour and data and can be deployed independently.

Typically represented as a rectangle with two smaller rectangles protruding from its sides, indicating ports for connections. The component's name is written inside the rectangle.

2. Artifact



An artifact represents a physical piece of information or data that is used or produced in the software development process. It can include source code files, executables, documents, libraries, configuration files, or any other item.

Typically represented as a rectangle with a folded corner, labelled with the artifact's name.

Artifacts may also include additional information, such as file extensions or versions.

3. Interface

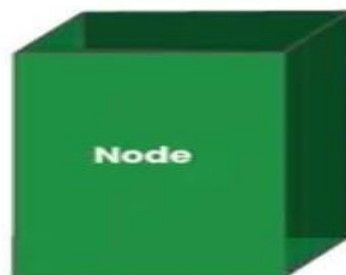


INTERFACE

An interface defines a contract specifying the methods or operations that a component must implement. It represents a point of interaction between different components or subsystems.

*Represented as a **circle or ellipse** labelled with the interface's name. Interfaces can also include provided and required interfaces, denoted by "+" and "-" symbols, respectively.*

4. Node

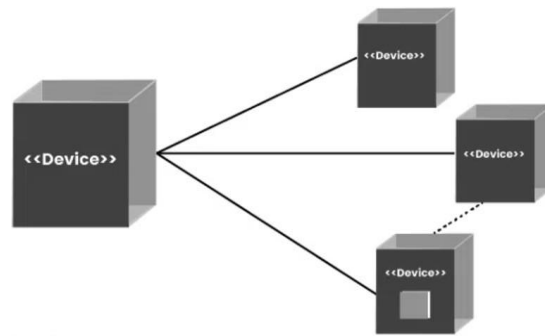


NODE

A node represents a physical or computational resource, such as a hardware device, server, workstation, or computing resource, on which software components can be deployed or executed.

*Represented as a box with **rounded corners**, usually labelled with the node's name. Nodes can also include nested nodes to represent hierarchical structures.*

5. Communication path



A straight line that represents communication between two devicenodes. Dashed lines in deployment diagrams represents relationships or dependencies between elements, indicating that one element is related to or dependent on another.

Deployment diagram of Easy Chat

